

Control flow

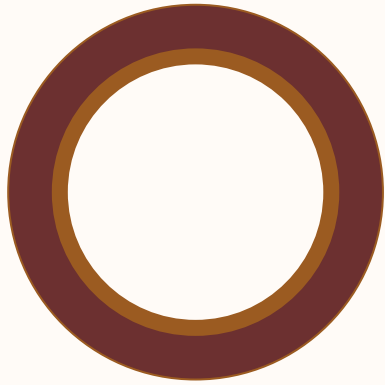
CS 22B, Spring 2026

Module 02: Control flow

Presented by Jessica Huynh-Westfall, Ph.D.

AGENDA

- Control flow
- Boolean logic
- if-elif-else
- Generator object
- Dictionary comprehension



Chapter 8: Control flow

Quick Python



Control flow refers to the order in which instructions are executed in a program. Typically from top to bottom, but not always.

Examples:

- A loop that repeats a section of code multiple times
- if/else execute block of code based on conditions
- Terminates or skip execute of certain lines of code
- Function calls another section of code, but remembers current place and local variables to be return

Control flow refers to the order in which instructions are executed in a program.

```
>>> for x in range(0, 3):  
...     print(x)  
...  
0  
1  
2
```



Control structure is the design of how flow is built. In this case it is the loop.



Control Flow is print(x)
print(x)
print(x)

Control Flow Statements Type	Control Flow Statement	Description
Conditional Statements	if-else	Executes a block of code if a specified condition is True, else another block if the condition is False.
Looping Statements	for	Executes a block of code a specified number of times, typically iterating over a range of values.
	while	Executes a block of code as long as a specified condition is true.
Jump Statements	break	Terminates the loop and transfers control to the statement immediately following the loop.
	continue	Skips the current iteration of a loop and continues with the next iteration.
	return	Exits a function and returns a value to the caller.

Class exercise

07

Boolean logic and evaluation

```
a = 4  
b = 7  
c = False
```

```
(a > 3 and b < 10) or c  
not (a == 4 and b != 7)  
(a >= 5) == (b <= 7)  
(not c and a < b) or (a == b)
```

CL 5.1

Without running Python, determine the value (True or False) for each expression. What is your reasoning for each?

if-elif-else statement

08

```
if condition1:
    body1
elif condition2:
    body2
elif condition3:
    body3
...
elif condition(n-1):
    body(n-1)
...
else:
    body(n)
```

- Condition is Boolean expression
- Body can include `pass` statement that serves as a placeholder but has no action

Class exercise

09

```
val = 5
d = 1
if val != d:
    v = val/val-5
    d += val+3
elif -5 < d:
    y = 3+val+d
    y -= 3-val
else:
    x = 2
    d += 4+val
print(d)
```

CL 5.2

Evaluate this if-elif-else, what will the code print?

Class exercise

10

```
x = 0
y = 10

if x != 0 and y / x > 2:
    result = "A"
else:
    result = "B"
print(result)
```

CL 5.3

1. *What is the value of result?*
2. *Will the code raise an exception (error when y/x)?*

for loop

11

for loop in Python iterates over the values returned by an iterable object (list, tuple, string, range, or generator).

```
for item in sequence: 
    body
else: 
    post-code
```

Item is element of sequence

else is optional

Using range in for loop - To specify explicit indices

Break and continue statements - special statement used in body of loop

Enumerate function in for loop

12

```
x = [1, 3, -7, 4, 9, -5, 4]
for i, n in enumerate(x):
    if n < 0:
        print("Found a negative number at index", i)
```

Enumerate function returns tuple of (index, item)

- You can access the items without the index (eg., `if n < 0:`)
- You can access the index (eg., `print(i)`)

List and dictionary comprehensions

13

Comprehension patterns

List comprehension

```
new_list = [expression1 for variable in old_list if expression2]
```

Dictionary comprehension

```
new_dict = {expression1:expression2 for variable in list if  
expression3}
```

Dictionary comprehension

14

```
x = [1, 2, 3, 4, 5]  
x_squared_dict = ??
```

Use a dictionary comprehension where the number in the list is the key and the number's square is the value in the dictionary.

Expected output

x_squared_dict is {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Class exercise

15

Given

```
words = ["data", "logic", "loop", "condition", "flow",  
"python"]
```

CL 5.6

Write a dictionary comprehension that maps each word to its length **only if** the word contains the letter 'o'.

Expected outcome

```
{"logic": 5, "loop": 4, ...}
```

Generator object

16

Use generator expression when we want to use an iterator in for loop. Generator object produces value one at a time when requested. Value is not stored in memory.

```
xs = [1, 2, 3, 4, 5]
xs_squared = (xs_item * xs_item for xs_item in x)
```

What will be printed when call xs_squared?

```
for square in xs_squared:
    print(square, )
```